

API Testing with Postman - Assignment Report

Student Name: Edith Amondi

Github Repository Link: https://github.com/lamAMONDI/Postman_API_testing

1. API CHOICE AND BASE URL

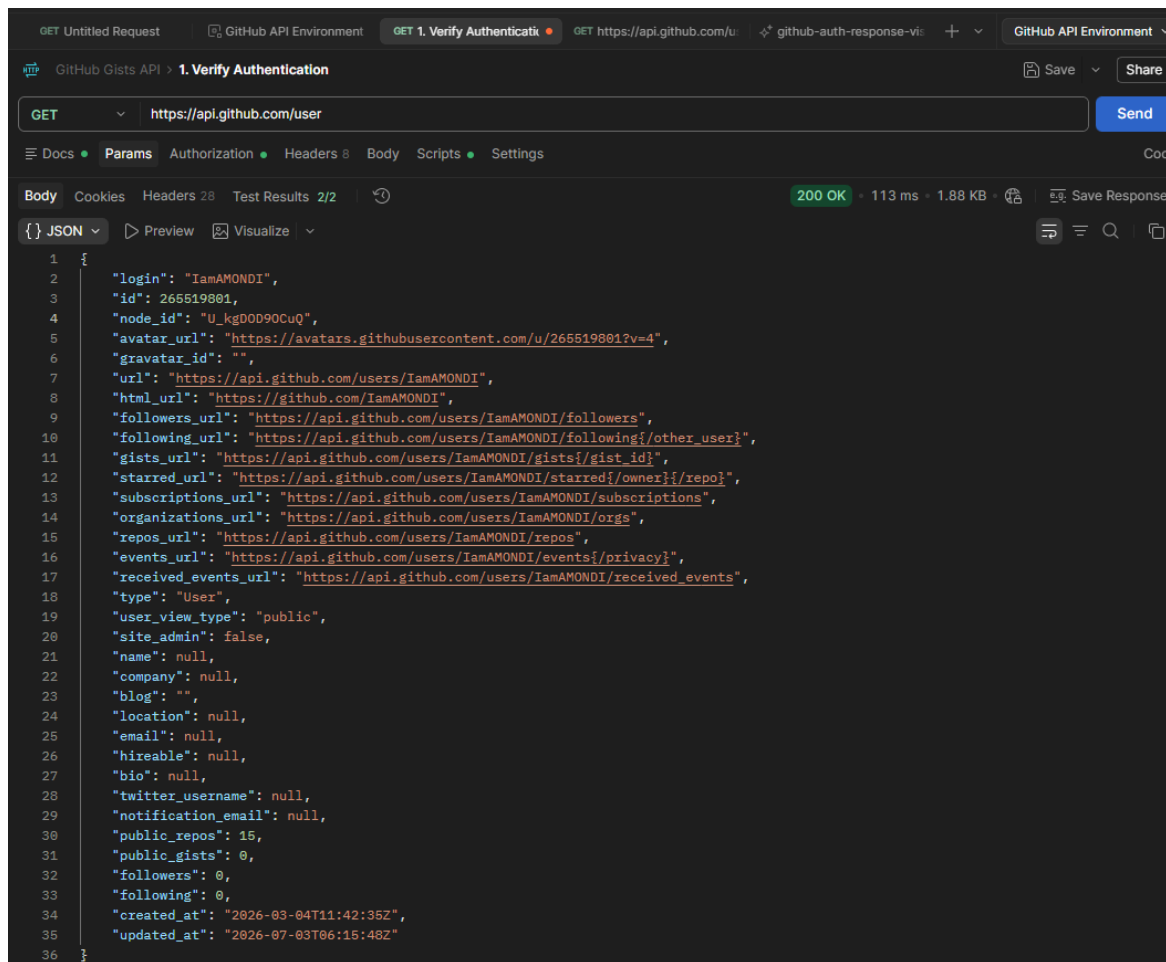
- API Choice: GitHub API (Gists)
- Base URL: <https://api.github.com>
- Reason for Choice: The GitHub API is an industry-standard, reliable public REST API. It supports the full CRUD lifecycle (Create, Read, Update, Delete) through Gists and implements real-world, secure token-based authentication (Personal Access Token). This was also very seamless to set up from classic token retrieval to adding it into environment variables in postman.

2. AUTHENTICATION METHOD USED

The GitHub REST API utilizes Personal Access Tokens (PAT) for user authentication. In Postman, authentication was configured at the Collection level using the "Bearer Token" type. The token itself is stored securely within the Postman Environment under the variable name `{{access_token}}`. This approach centralizes authorization so every request automatically inherits the token without needing manual header configuration. Initially I tried the requests without having the token in place. It was better to have it inherited.

Note: Unlike APIs that provide a `/login` endpoint to exchange username/password for a temporary session token, GitHub PATs are provisioned securely via GitHub's Developer Settings. A dedicated GET request to the `/user` endpoint was implemented to validate that the token is active and authorized before running subsequent operations.

Figure 1: Verify Authentication - ``GET /user`` - ``200 OK`` response. This screenshot shows the GET request made to verify that the classic Personal Access Token generated from my GitHub account is valid and authorized. The response body returns authenticated user details confirming successful Bearer Token authentication.

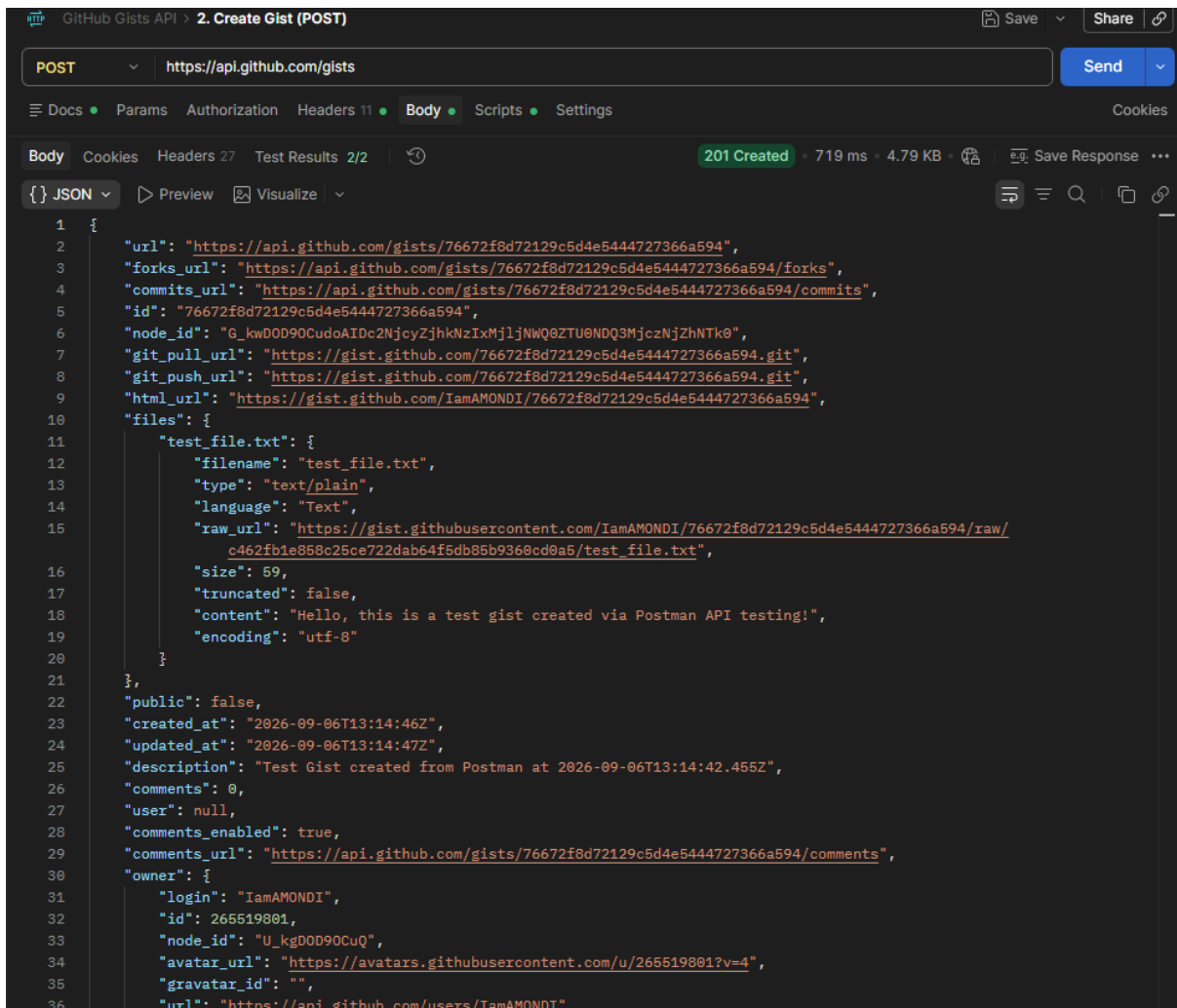


3. CRUD OPERATIONS BREAKDOWN

a. CREATE (POST)

- Endpoint: POST /gists
- Purpose: Creates a new secret Gist on the authenticated GitHub account.
- Pre-request Script: Executes JavaScript to dynamically generate a timestamp and stores it in the environment variable "current_timestamp", ensuring every run creates a uniquely identifiable description.
- Test Script: Asserts that the HTTP response status code is 201 Created. Parses the response JSON body, extracts the new unique id, and dynamically stores it into the Postman environment variable "gist_id" to chain into the next requests.

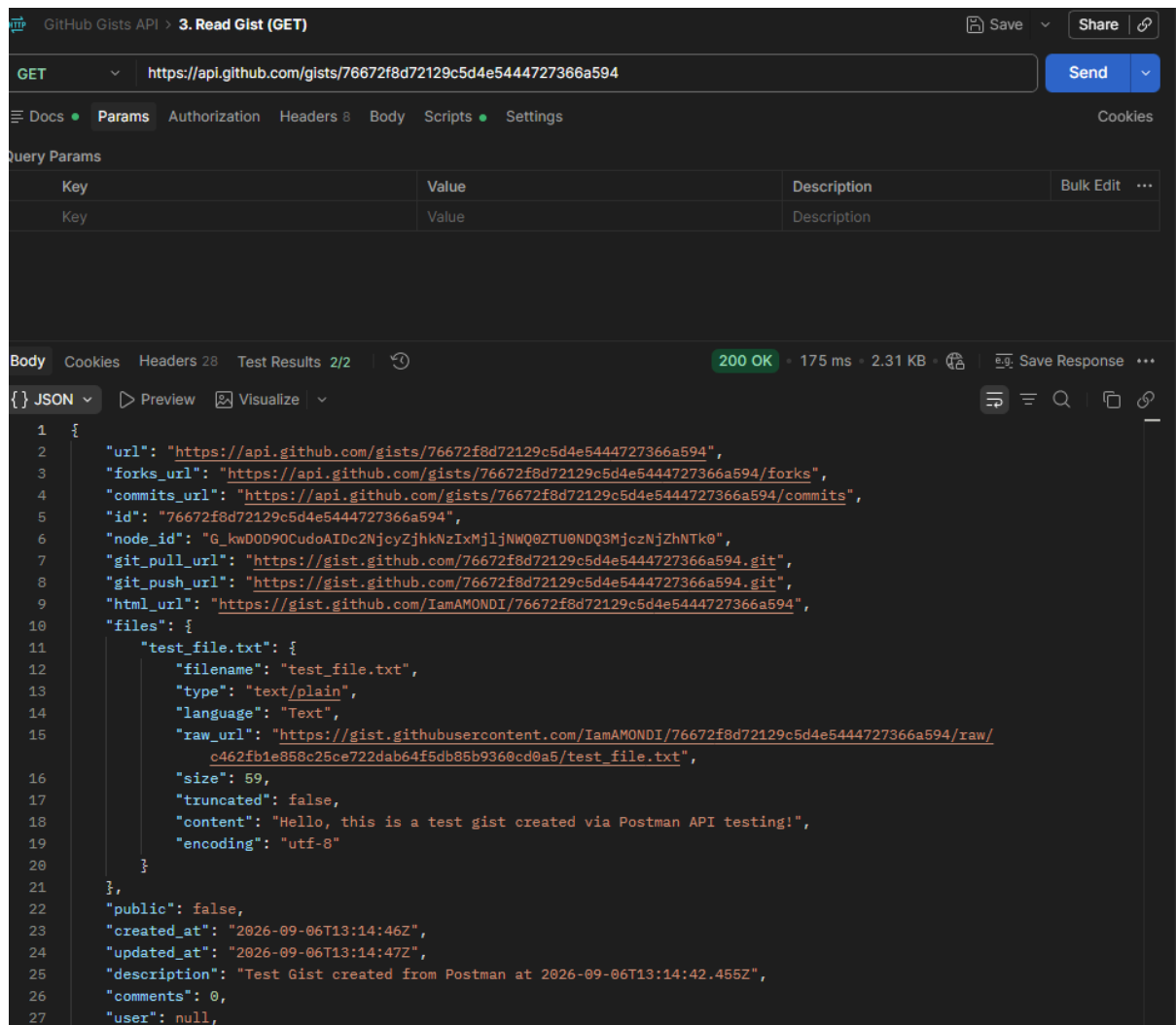
Figure 2: Create Gist - `POST /gists` - `201 Created` response. This screenshot shows the successful creation of a new secret test Gist. The test script automatically extracts the new Gist's unique `id` from the response body and saves it into the `{{gist\_id}}` environment variable for use in subsequent requests.



b. READ (GET)

- Endpoint: GET /gists/{{gist_id}}
- Purpose: Fetches the specific Gist created in the POST step using the dynamic ID.
- Test Script: Asserts that the HTTP response status code is 200 OK and validates that the returned "id" matches the "gist_id" stored in the environment.

Figure 3: Read Gist - `GET /gists/{{gist\_id}}` - `200 OK` response. This screenshot shows the successful retrieval of the Gist created in Figure 2. The test script validates that the returned `id` matches the `gist\_id` stored in the environment variable, confirming accurate request chaining.



c. UPDATE (PATCH)

- Endpoint: PATCH /gists/{gist_id}
- Purpose: Updates both the description and file content of the existing Gist.
- Request Body (JSON):

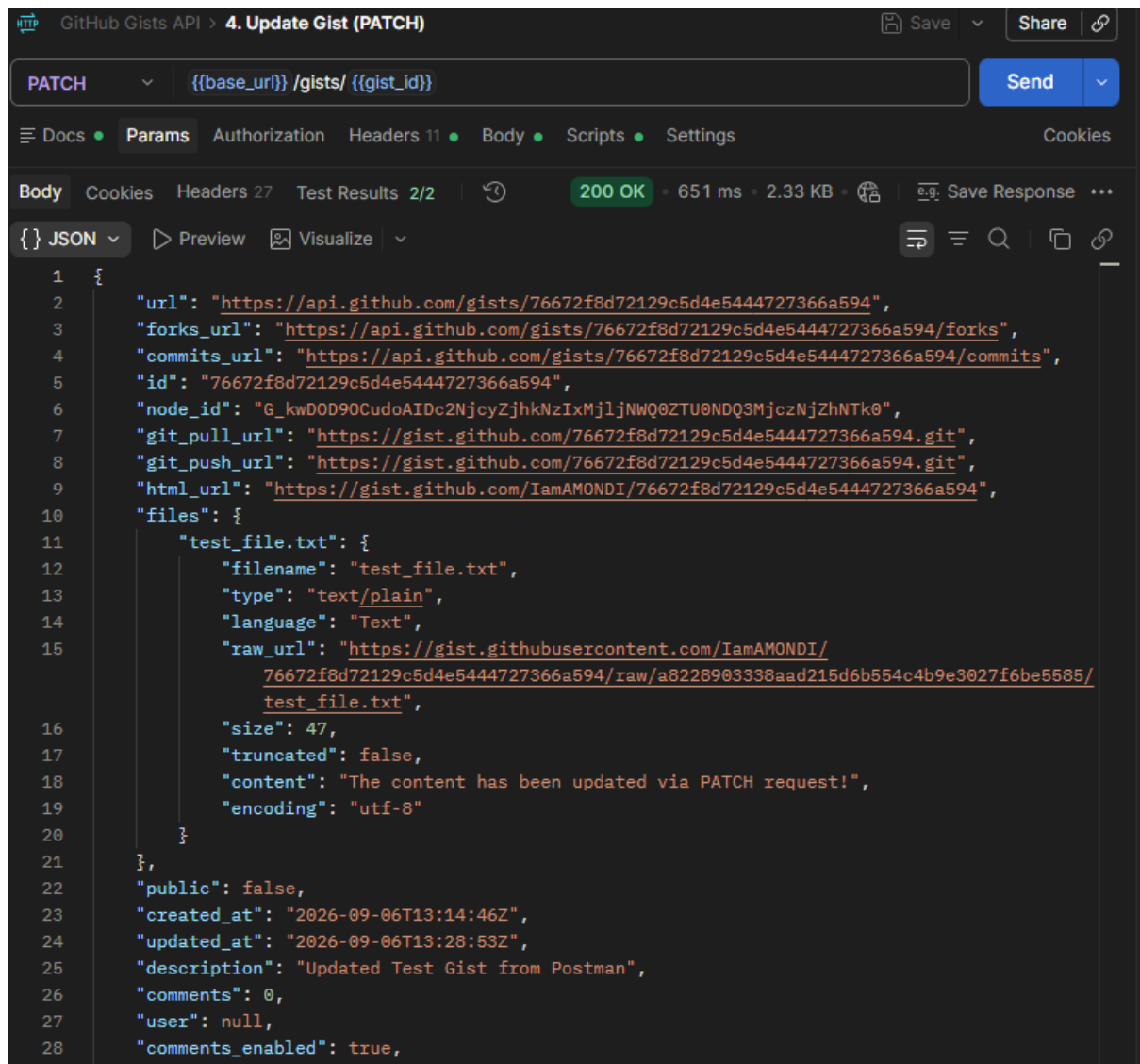

```

{
  "description": "Updated Test Gist from Postman",
  "files": {
    "test_file.txt": {
      "content": "The content has been updated via PATCH request!"
    }
  }
}

```
- Test Script: Asserts that the HTTP response status code is 200 OK and validates that the description field returned in the response matches the new updated text.

Figure 4: Update Gist - `PATCH /gists/{gist\_id}` - `200 OK` response. This screenshot shows the successful PATCH request updating the Gist's description and file content. The

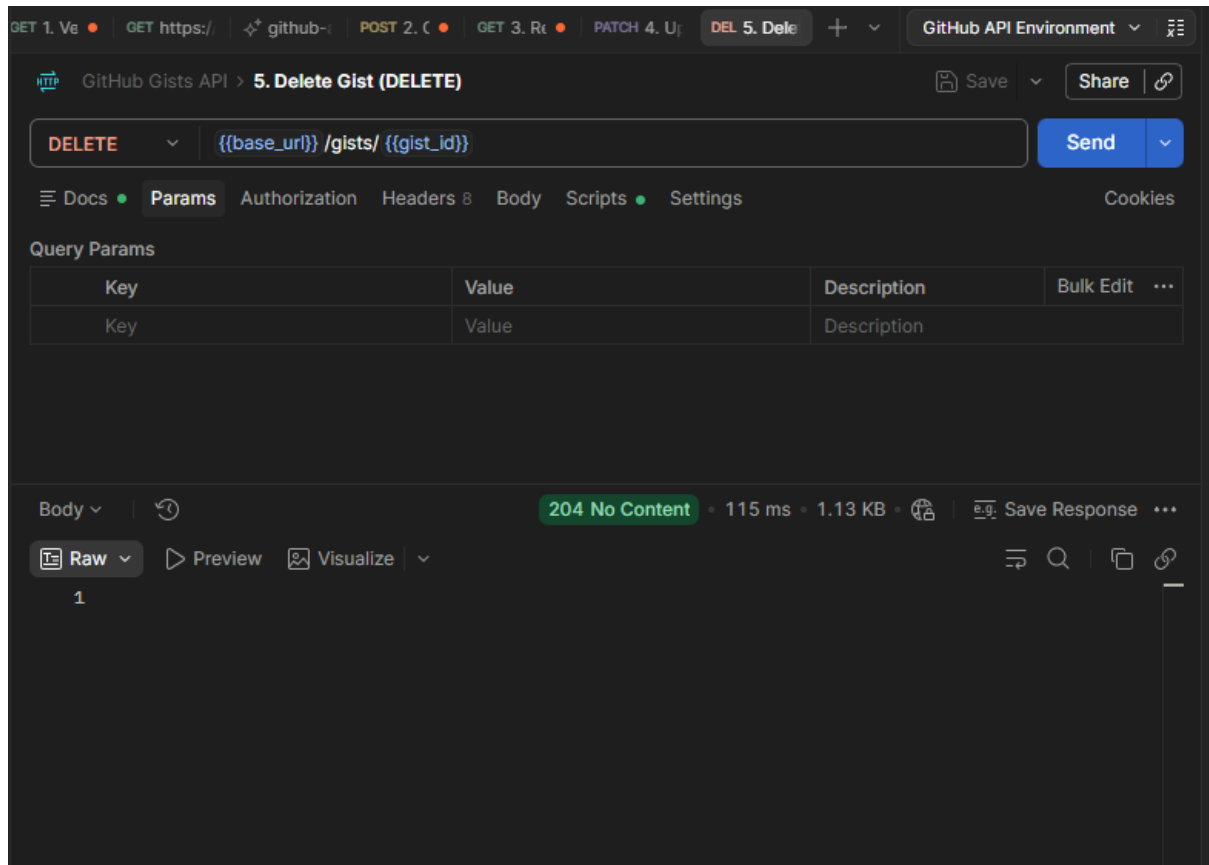
test script asserts that the description field in the response matches the new updated text:
"Updated Test Gist from Postman"



d. DELETE (DELETE)

- Endpoint: DELETE /gists/{{gist_id}}
- Purpose: Permanently deletes the test Gist to clean up resources on the server.
- Test Script: Asserts that the HTTP response status code is 204 No Content, confirming successful deletion with no body content returned.

Figure 5: Delete Gist - `DELETE /gists/{gist_id}` - `204 No Content` response. This screenshot confirms the successful deletion of the Gist created in Figure 2. The `204 No Content` status code indicates the server processed the request and the resource has been permanently removed, with no response body returned.



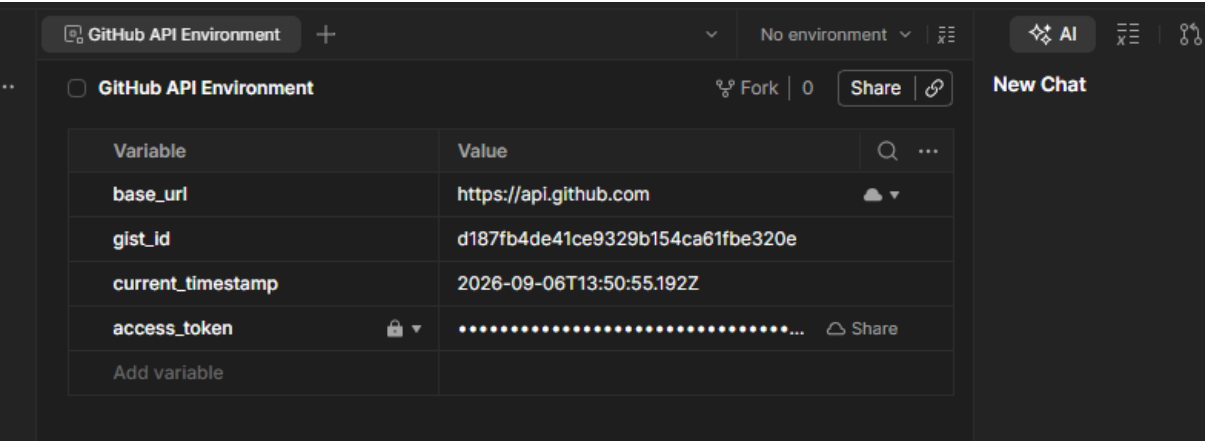
4. ENVIRONMENT VARIABLES AND AUTOMATION

To make the collection automated, dynamic, and reusable without hardcoding values, the following Postman environment variables were utilized:

1. `base_url`: Stores "https://api.github.com" so endpoints do not need repeating.
2. `access_token`: Securely holds the GitHub Personal Access Token (PAT).
3. `gist_id`: Automatically captured from the response of the POST request and passed dynamically into the GET, PATCH, and DELETE requests.
4. `current_timestamp`: Generated in the pre-request script and injected into the request.

This architecture enables request chaining, eliminating the need to manually copy and paste IDs between requests.

Figure 6: This screenshot shows the postman editor for the environment variables I added.



5. COLLECTION RUNNER EXECUTION

The entire collection was executed automatically from start to finish using the Postman Collection Runner. All 5 requests executed sequentially, and all automated test assertions passed with green status indicators.

Figure 7: Collection Runner - Failed Run. This screenshot shows how the initial Collection Runner execution failed because the access token had not been correctly saved into both the Initial Value and Current Value fields of the environment variables, causing all requests to return `401 Unauthorized`

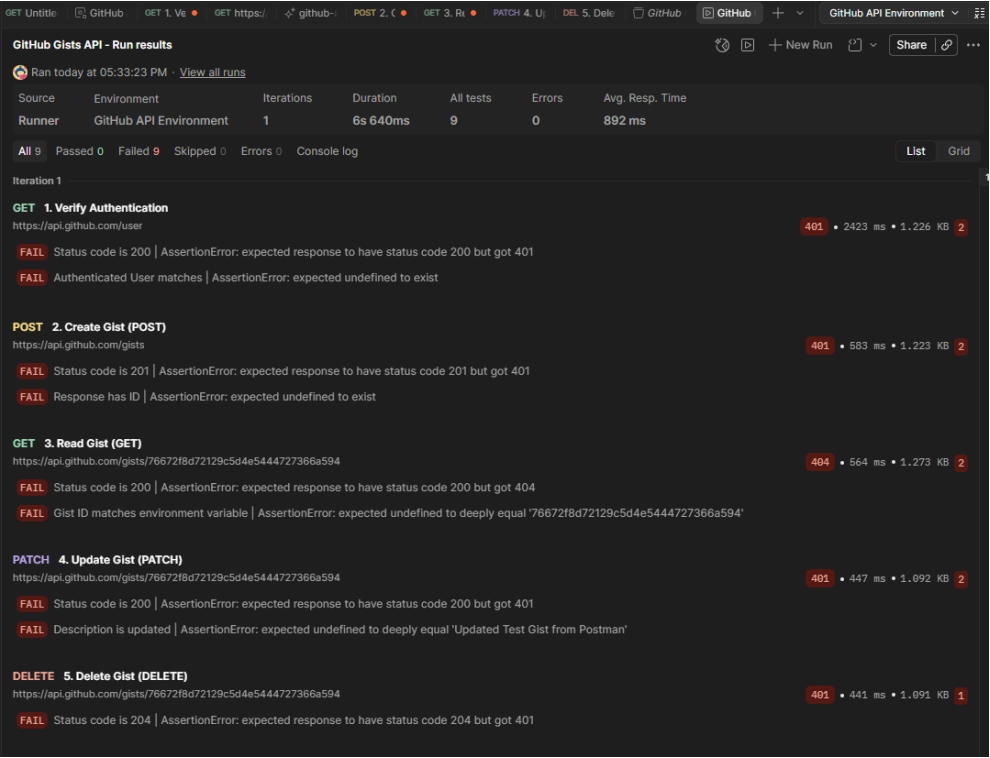
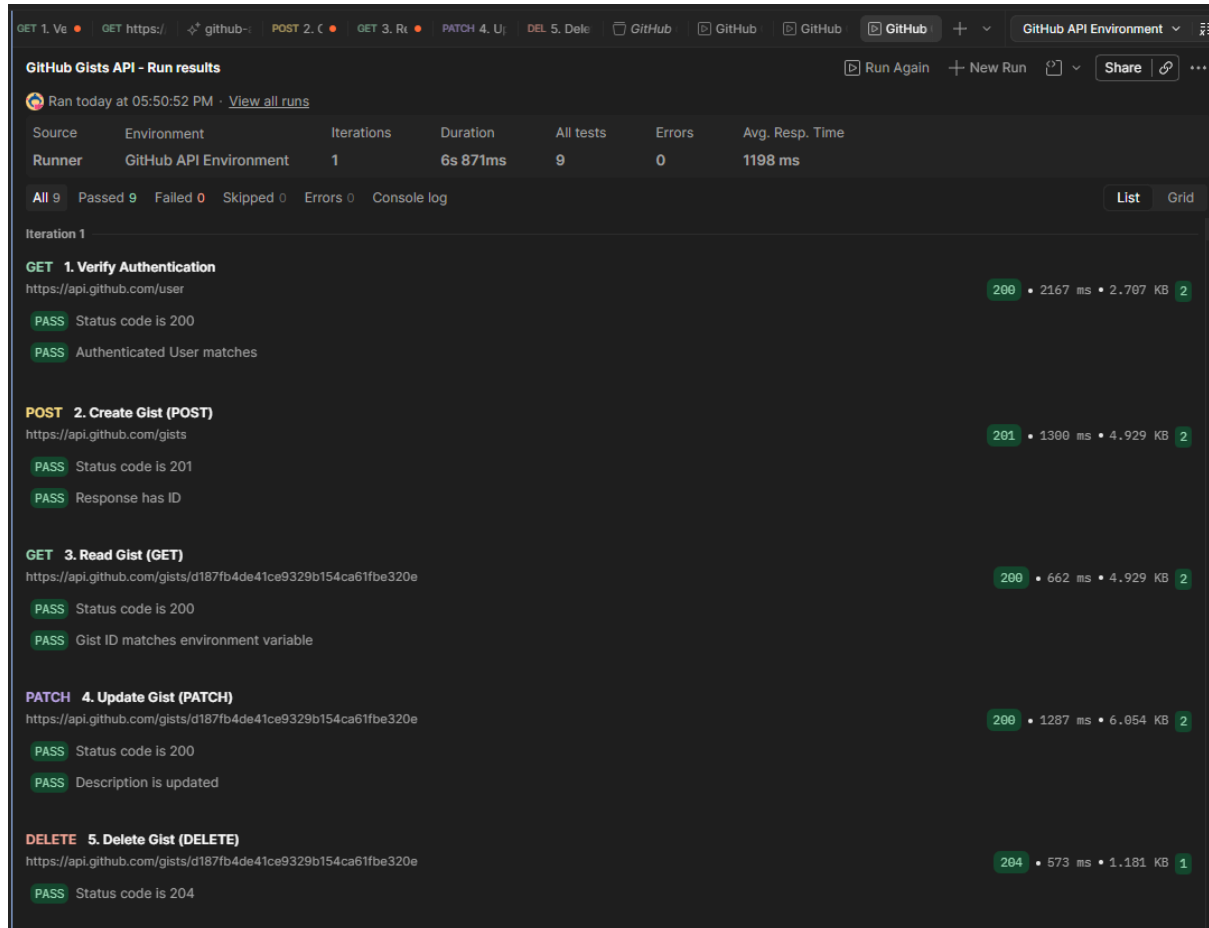


Figure 8: Collection Runner - Successful Run - All 9/9 Tests Passed. This screenshot shows the successful Collection Runner execution after fixing the environment variable configuration. All 5 requests executed sequentially with all 9 automated test assertions passing with green status indicators, demonstrating a fully automated end-to-end API testing workflow.



6. ASSIGNMENT REFLECTION

CHALLENGES

1. Environment Variable Configuration (Initial vs Current Value):

My first Collection Runner execution failed with all 9 tests returning `401 Unauthorized`. This was because I had only pasted my GitHub Personal Access Token into the "Current Value" field of the Postman environment, but not the "Initial Value" field. The Collection Runner reads from the Initial Value, while individual request tabs read from the Current Value. Once I pasted the token into both fields and saved, all tests passed successfully.

2. Importing the Collection into Postman.

When I first opened Postman, the Collections tab appeared empty even after importing the JSON files. I had to navigate manually through File Explorer to locate the `.postman\_collection.json` and `.postman\_environment.json` files, as they were stored in a

hidden directory on my system. Once I located and re-imported them correctly, the collection appeared.

3. Understanding Bearer Token Authentication:

Initially, I was unsure about the relationship between the GitHub Classic Token and the Bearer Token in Postman. I learned that the Classic Token (`ghp\_...`) is the actual credential, while "Bearer Token" is the HTTP authentication scheme that tells the server how the token is being transmitted in the `Authorization` header.

What I Learned

1. Request Chaining with Environment Variables:

I learned how to use Postman's test scripts to dynamically capture data from one API response (such as the `gist\_id` from the POST request) and store it in environment variables so that subsequent requests (GET, PATCH, DELETE) can use it automatically without manual copy-pasting.

2. The Importance of Sequential Execution:

I learned that when API requests depend on each other (e.g., DELETE depends on the ID created by POST), they must be executed in the correct order. Running them out of sequence causes failures because required variables have not yet been populated.

3. Collection-Level Authentication.

I discovered that configuring authentication at the Collection level, rather than on each individual request, is a best practice. It centralizes credentials, reduces repetition, and ensures consistency across all requests.